

name:Madhumitta Ch S
Reg no:1921260101

1. Explain te difference between Multiplexing and Demultiplexing with a real-life example. How do TCP sockets use the 4-tuple to achieve demultiplexing?

features	multiplexing	demultiplexing
meaning	Combines data from multiple applications/sources into one communication channel	Separates incoming data and delivers it to the correct application
direction	Sender side	Receiver side
Main purpose	Efficiently share a network connection	Correctly identify the destination application
example	Many phone calls sharing the same telephone network	Network directing each call to the correct person's phone

Real-life example: Highway

Imagine a **highway with vehicles from different cities**:

- **Multiplexing:** Vehicles from different starting points enter the same highway. The highway carries all of them together.
- **Demultiplexing:** At different exits, vehicles leave the highway and go to their respective destinations.

Similarly, a computer can run **web browsing, email, and online gaming simultaneously**. Their data travels through the same network connection, but the receiver must deliver each piece of data to the correct application.

How TCP uses the 4-tuple for demultiplexing

TCP identifies a connection using a **4-tuple**:

4-tuple = (Source IP address, Source Port, Destination IP address, Destination Port)

For example:

(192.168.1.10, 5000, 142.250.195.14, 443)

This means:

- **Source IP:** 192.168.1.10 → sender's computer
- **Source Port:** 5000 → port used by the sender application
- **Destination IP:** 142.250.195.14 → destination server
- **Destination Port:** 443 → HTTPS service

When TCP receives a segment, it examines these values and finds the **matching socket/connection**. It then delivers the data to the correct application.

Simple flow

Multiple applications

↓

Multiplexing

↓

Network

↓

TCP checks 4-tuple

↓

Demultiplexing

↓

Correct application/socket

In short:

Multiplexing = many applications → one network connection

Demultiplexing = one network connection → correct applications

The TCP 4-tuple allows the system to distinguish different TCP connections, even when several connections use the same server IP and destination port.

2. Compare different traffic management techniques (e.g., traffic shaping vs traffic policing). Which one is better for real-time applications like video calls and why?

Traffic Management Techniques

Traffic management controls how network packets are transmitted so that the network avoids congestion and provides better performance.

1. Traffic Shaping

Traffic shaping **controls the rate at which packets are sent**. It usually stores excess packets in a buffer and sends them gradually at a controlled rate.

Example:

If a network allows 10 Mbps but an application produces data at 15 Mbps, traffic shaping can temporarily buffer the extra traffic and transmit it smoothly at about 10 Mbps.

Advantages:

- Smooths sudden traffic bursts.
- Reduces congestion.
- Provides more predictable traffic flow.

Disadvantage:

- Buffering can introduce **delay**, especially when the network is busy.

2. Traffic Policing

Traffic policing **monitors the traffic rate** and enforces a specified limit. If traffic exceeds the limit, excess packets may be **dropped or marked**.

Example:

If the allowed rate is 10 Mbps and traffic reaches 15 Mbps, the extra traffic may be discarded or marked as lower priority.

Advantages:

- Prevents a user/application from exceeding its allocated bandwidth.
- Does not require large buffering.
- Helps enforce network policies.

Disadvantages:

- Dropping packets can cause retransmissions or loss.
- Sudden packet loss can affect applications that require continuous data.

features	Traffic shaping	Traffic policing
Main action	Delays/buffers excess packets	Drops or marks excess packets
Traffic flow	smooth	Can be abrupt
Packet loss	Usually lower	Can be higher
delay	May increase	Usually lower from buffering
Best use	Smoothing burst	Enforcing bandwidth limit

Which is better for video calls?

Traffic shaping is generally better for real-time applications such as video calls, provided it is configured carefully.

Video calls require:

- Low packet loss
- Low and consistent delay
- Low jitter
- Sufficient bandwidth

Shaping can smooth traffic bursts instead of immediately dropping packets, which can help maintain continuous audio and video.

However, **excessive shaping is also harmful** because waiting in a buffer increases delay. For video calls, the best approach is usually **QoS with priority treatment for real-time traffic**, combined with carefully configured shaping/policing for other traffic.

Conclusion:

👉 Traffic shaping is preferable for smoothing traffic, while traffic policing is better for enforcing bandwidth limits. For video calls, controlled shaping + QoS priority is usually the better choice because real-time traffic is highly sensitive to packet loss, delay, and jitter.

3. Compare Single Server and Multi-server queuing models. In which network scenario would a multi-serverqueue perform better?

Single-Server vs Multi-Server Queuing Models

A queuing model describes how requests or packets wait for service when a network resource is busy.

Features	Single server queue	Multi server queue
Number of serves	one	Two or more
Service capacity	Limited to one server	High because multiple server work simultaneously
Waiting time	Usually higher	Usually lower
Queue length	Can become long	Generally short
Cost	lower	higher
example	One router/server handling requests	Multiple routers/servers handling requests

Single-Server Model

In a **single-server queue**, all incoming requests wait in one queue and are processed by **one server**.

Example:

A network has one server handling all web requests. If many users send requests simultaneously, some requests must wait until the server becomes free.

Flow:

Requests → Queue → One Server → Output

Multi-Server Model

In a **multi-server queue**, several servers can process requests at the same time. Incoming requests can be assigned to any available server.

Example:

A cloud service has several servers handling thousands of user requests. If one server is busy, a new request can be sent to another available server.

Flow:

Requests → Queue → Server 1 / Server 2 / Server 3 → Output

Which performs better in a network?

A **multi-server queue** performs better when network traffic is heavy and many requests arrive simultaneously.

For example, consider a **busy web service or cloud data center**:

- Single server → requests build up in a long queue.
- Multiple servers → requests are distributed among available servers.
- Therefore, **waiting time and congestion are reduced** and more requests can be processed simultaneously.

Conclusion

Single-server: Simple and inexpensive, but suitable mainly for low or moderate traffic.

Multi-server: More expensive but provides **higher processing capacity, shorter waiting time, and better performance under heavy network traffic**.

👉 Therefore, a multi-server queue is preferable for high-traffic networks such as cloud data centers, large web servers, and busy network service systems.